

It may seem that gradient ascent is overcomplicating the issue, and that a guess-and-check strategy is sufficient for finding the maximum. A guess-and-check strategy would consist of guessing a potential maximum and checking whether it is higher than all previously guessed candidate maxima until we are confident that we have found the maximum. One potential reply to this is to point out that, just as with visual inspections, with high-complexity multidimensional functions, guess-and-check could be prohibitively difficult to successfully implement in practice. But the best reply to the idea of guessing and checking to find maxima is that this is exactly what gradient ascent is *already doing*. Gradient ascent already is a guess-and-check strategy, but one that is “guided” by moving guesses in the direction of the gradient rather than by guessing randomly. Gradient ascent is just a more efficient version of guess-and-check.

Finally, consider the idea of solving the first-order conditions to find a maximum. This is a method that is taught in calculus classes all around the world. It could be called an algorithm, and its steps are:

1. Find the derivative of the function you are trying to maximize.
2. Set that derivative equal to zero.
3. Solve for the point at which the derivative is equal to zero.
4. Make sure that point is a maximum rather than a minimum.

(In multiple dimensions, we can work with a gradient instead of a derivative and perform an analogous process.) This optimization algorithm is fine as far as it goes, but it could be difficult or impossible to find a closed-form solution for which a derivative is equal to zero (step 2), and it could be harder to find that solution than it would be to simply perform gradient ascent. Besides that, it could take huge computing resources, including space, processing power, or time, and not all software has symbolic algebra capabilities. In that sense, gradient ascent is more robust than this algorithm.